

作业一：文本分类实现

一、实验目标

本次作业旨在帮助大家理解和掌握不同文本表示方法在文本分类任务中的基本原理与实际效果。你将分别使用传统词袋模型、Word Embedding / Word2Vec，以及预训练语言模型对文本进行表示，并训练文本分类模型。

1. 理解不同文本表示方法的基本思想；
2. 掌握文本分类任务的基本实验流程；
3. 熟悉 Logistic Regression 在文本分类中的使用；
4. 掌握 Word2Vec、GloVe 等词向量的基本使用方法；
5. 熟悉 BERT 类预训练语言模型的 Fine-tuning 流程；
6. 能够使用 Accuracy 和 Macro-F1 对文本分类模型进行评价；
7. 能够对不同文本表示方法的性能进行基本比较和分析。

二、NYT 数据集划分

题目要求统一设置：

数据集	比例
Training Set	80%
Validation Set	10%
Test Set	10%

其中，NYT 数据集包含 text 和 label 两列，对于它的 label，经打印，有：

Label	数量
sports	8639
politics	1451
business	1429

为了使数据集能够被合理且无偏地划分为训练集、验证集和测试集，同时保持各个 label 的比例一致，采取如下的代码：

```
train_df, temp_df = train_test_split(
    df,
    test_size=0.2,
    random_state=42,
    shuffle=True,
    stratify=df["label"]
)
```

```
val_df, test_df = train_test_split(
    temp_df,
    test_size=0.5,
    random_state=42,
    shuffle=True,
    stratify=temp_df["label"]
)
```

其中 `stratify=df["label"]` 是为保证各 label 的分布比例与原始数据集保持一致。

为了对所有实验均应使用相同的数据划分，最后对对应数据进行储存。

我在开始的时候错误地将文件命名为 `select.py`，但是由于 Python 标准库中存在同名模块，导致在执行 `import select` 时错误地引用了自己出现了循环引用问题。修改文件名后得以解决。

三、Task 1: Bag of Words

使用两种 Bag-of-Words 文本表示方法训练文本分类器。它完全忽略了词语的顺序、语法和句法关系，只关注词语本身的出现情况。

1. Binary Bag of Words

本方法只考虑一个单词是否出现，而不考虑它出现了多少次。

使用已有的 `CountVectorizer` 函数进行文本特征提取与向量化。它默认会进行大小写转换，如下：

```
vectorizer = CountVectorizer(
    binary=True
)

X_train = vectorizer.fit_transform(
    X_train_text
)

X_test = vectorizer.transform(
    X_test_text
)
```

2. Word Frequency

它不仅关注词语是否出现，还关注词语出现的次数。

具体实现是：

```
vectorizer = CountVectorizer()
```

3. 实验结果

方法	Accuracy	Macro-F1
Binary Bag of Words	99.0451%	97.6383%
Word Frequency	98.9583%	97.4178%

实验结果表明，Binary Bag of Words 和 Word Frequency 两种传统词袋表示方法均能够在 NYT 新闻分类任务中取得较高性能。其中 Binary Bag of Words 在 Accuracy 和 Macro-F1 两项指标上均略优于 Word Frequency。说明对于该新闻分类任务而言，关键词是否出现的信息已经能够有效区分不同类别，而词语重复次数带来的额外信息有限。同时，由于新闻类别之间存在明显的关键词差异，简单的词袋表示结合 Logistic Regression 即可获得较好的分类效果。

四、Task 2: Word2Vec (50 分)

本部分使用基于 Word Embedding 的方式表示文档。也就是将文档中每个单词映射到一个能够捕捉语义相似度的高维向量空间中，再将这些词向量聚合为一整篇文档的表达。

1. Pre-trained GloVe

使用公开的预训练 GloVe Embedding (glove.6B)，并选择 100-dimensional GloVe word vectors 作为单词表示。

对于一篇包含 n 个有效单词的文档，文档向量就是所有有效向量的平均值。

$$e(d) = (1 / n) \times \sum_{i=1}^n e(w_i)$$

对应代码核心逻辑也较为简单，如下：

```
for word in words:
    if word in glove:
        vectors.append(
            glove[word]
        )
if len(vectors)==0:
    return np.zeros(100)
return np.mean(
    vectors,
    axis=0
)
```

也就是先对文本进行小写化，然后进行分词，最后计算出文档向量。

过程中发现 glove.6B.100d.txt 文件体积过大，在上传 github 时候需要选择忽略。

2. Word2Vec Trained on AG News

使用 AG News 的文本数据自行训练一个 100-dimensional Word2Vec Model，然后对每一篇 NYT 文档，重复 1.中的操作。

关键代码如下：

```
w2v_model = Word2Vec(  
    sentences=ag_sentences,  
    vector_size=100,  
    window=5,  
    min_count=2,  
    workers=4  
)
```

其中，ag_sentences 是 AG News 数据集中每一篇文章被切分好的单词列表。

3. Word2Vec Trained on NYT

相比于 2.，只需要进行如下修改：

```
w2v_model = Word2Vec(  
    sentences=nyt_sentences,  
    vector_size=100,  
    window=5,  
    min_count=2,  
    workers=4  
)
```

4. 实验结果

方法	Accuracy	Macro-F1
Pre-trained GloVe	98.5243%	96.4617%
Word2Vec Trained on AG News	97.7431%	94.5803%
Word2Vec Trained on NYT	98.3507%	95.7955%

其中，Pre-trained GloVe 取得最高性能。这是由于 GloVe 在大规模通用语料上进行预训练，通过词共现信息学习词语之间的语义关系，因此能够获得较好的通用语义表示，

在 NYT 新闻分类任务中也能够有效捕捉不同类别新闻中的关键词和语义特征。

Word2Vec Trained on NYT 的性能高 Word2Vec Trained on AG News，是因为前者直接利用 NYT 训练集中的新闻文本学习词向量，因此词向量更加贴合当前任务领域。

五、Task 3: Pre-trained Neural Models (20 分)

使用预训练语言模型直接完成文本分类任务。需要微调 BERT，并在 NYT Test Set 上报告 Accuracy 和 Macro-F1。

核心代码如下：

```
model = BertForSequenceClassification.from_pretrained(
    "google-bert/bert-base-uncased",
    num_labels=3
)
```

```
encoding = self.tokenizer(
    text,
    max_length=64,
    padding="max_length",
    truncation=True,
    return_tensors="pt"
)
```

```
output = model(
    input_ids,
    attention_mask=mask,
    labels=labels
)
loss = output.loss
loss.backward()
optimizer.step()
```

```
output = model(
    input_ids,
    attention_mask=mask,
)
prediction=torch.argmax(
    output.logits,
    dim=1
)
```

实验结果如下：

epoch	total_loss
1	83.6342
2	24.3128
3	11.5542

随着训练轮数增加，模型的训练损失持续下降。说明 BERT 模型参数在微调过程中不断适应 NYT 新闻分类任务，模型逐渐学习到了不同新闻类别之间的语义表示特征。

最终有：

Accuracy	Macro-F1
98.2639%	96.1238%

表明 BERT 能够较好完成新闻文本分类任务。

六、实验总结

本次实验完成了从词袋模型、平均词向量到 BERT 微调的三组文本分类任务。六种方法的测试准确率均达到了 97.7% 以上，其中 Binary Bag of Words 取得了最高的 Accuracy (99.0451%) 和 Macro-F1 (97.6383%)。在静态词向量方法中，Pre-trained GloVe 的效果最好 (98.5243%)。微调后的 BERT 模型训练耗费了最多的时间，但仅在 Macro-F1 表现上优于 Word2Vec Trained on AG News 与 Word2Vec Trained on NYT。

整体来看，模型更复杂并不意味着在此特定任务中一定表现更好，不同类型的数据适配不同类型的模型。词袋模型虽然简单，但已能极为高效地利用新闻中的关键词特征。BERT 模型在较少的训练轮数上表现可能不如普通模型亮眼。

通过本次实验，我对不同文本表示方式的区别有了更具体的认识。词袋模型直接基于词语出现与否或词频建立高维表示，平均词向量用较低的维度压缩整篇文档，BERT 则在微调中学习分类任务强相关的深层上下文特征。另外，NYT 数据集中体育新闻的数量明显多于政治和财经，因此除了看总体准确率，更需要结合 Macro-F1 来全面评估模型在小类上的表现。